



AI / ML BYOP COURSE PROJECT

AI-Based Badminton Shot Prediction System

Predicting the optimal shot — Smash, Drop, or Clear — from real-time game conditions using a Decision Tree Classifier

Project Report

Submitted as part of the AI/ML Build-Your-Own-Project (BYOP) requirement

Repository: Vityarthi_AIML_project | **Primary Algorithm:** Decision Tree Classifier

Tech Stack: Python, scikit-learn, pandas, Streamlit, Matplotlib

Report generated September 2026

Table of Contents

1. Introduction	3
2. Problem Statement	3
3. Functional Requirements	4
4. Non-Functional Requirements	4
5. System Architecture	5
6. Design Diagrams	6
7. Design Decisions & Rationale	10
8. Implementation Details	11
9. Screenshots / Results	13
10. Testing Approach	16
11. Challenges Faced	17
12. Learnings & Key Takeaways	17
13. Future Enhancements	18
14. References	18

1. Introduction

Badminton is one of the fastest racquet sports in the world, and shot selection during a rally is a critical, split-second decision that separates skilled players from beginners. A player must simultaneously read their own court position, the opponent's position, the difficulty of the incoming shuttle, and the pressure of the current score — and then choose between an aggressive **Smash**, a deceptive **Drop**, or a defensive **Clear**.

This project, the **AI-Based Badminton Shot Prediction System**, was built as a Build-Your-Own-Project (BYOP) submission for an AI/ML course. It demonstrates a complete, end-to-end supervised machine learning pipeline: synthetic data generation, exploratory data analysis, model training and evaluation, model persistence, and deployment behind an interactive web interface. The chosen algorithm, a **Decision Tree Classifier**, was selected specifically for its interpretability — an important property in a coaching context, where understanding *why* a shot is recommended matters as much as the recommendation itself.

The repository contains a dataset generator, a Jupyter notebook covering the full model-development workflow, and a Streamlit web application that lets a user interactively test the model with adjustable court positions, score situations, and shuttle difficulty levels.

2. Problem Statement

Beginner and intermediate badminton players often lack a structured way to practice shot-selection decision-making outside of live match play, and coaches lack a simple, visual tool to illustrate how court geometry and match context should influence shot choice. There is a need for a lightweight, accessible system that can take a description of a rally situation and recommend the most tactically sound shot, along with an explanation of the reasoning and the model's confidence in that recommendation.

Goal: Given a player's position, an opponent's position, the shuttle's difficulty, and the score situation, predict the most appropriate badminton shot — Smash, Drop, or Clear — and present this recommendation through an interactive, visual web interface.

3. Functional Requirements

ID	Requirement
FR-1	The system shall generate a synthetic dataset of badminton rally scenarios using a documented, rule-based shot-selection heuristic.
FR-2	The system shall allow exploratory analysis of the dataset, including class distribution and feature-vs-target relationships.
FR-3	The system shall encode categorical features (Score Situation, Shuttle Difficulty, Recommended Shot) into numeric form suitable for model training.
FR-4	The system shall train a Decision Tree Classifier to predict the recommended shot from six input features.
FR-5	The system shall evaluate the trained model using accuracy, precision, recall, F1-score, and a confusion matrix.
FR-6	The system shall persist the trained model and all label encoders to disk so they can be reloaded without retraining.
FR-7	The system shall provide a web-based UI allowing a user to set player and opponent court coordinates via sliders.
FR-8	The system shall allow a user to select a score situation (Early / Mid / Pressure) and a shuttle difficulty (Easy / Medium / Hard) from dropdown menus.
FR-9	On request, the system shall predict and display the recommended shot along with its class-wise confidence scores.
FR-10	The system shall render a visual court diagram showing the player and opponent positions relative to the net.
FR-11	The system shall display a short, plain-language tactical explanation for each recommended shot type.

4. Non-Functional Requirements

Category	Requirement
Usability	The interface must be operable without any machine-learning background — all inputs are simple sliders and dropdowns with sensible defaults.
Performance	A prediction must be returned interactively (sub-second) after the user clicks "Predict Best Shot", since the Decision Tree offers O(depth) inference.
Interpretability	The underlying model must support inspection (feature importance, visualized tree, per-class probabilities) rather than behave as a black box.
Portability	The application must run on any machine with Python 3.8+ and the packages listed in <code>requirements.txt</code> , without external services or paid APIs.
Maintainability	Data generation, model training, and the UI must be separated into distinct, independently runnable scripts/notebooks.
Reproducibility	All randomized steps (dataset generation, train/test split, model training) must use a fixed random seed (42) for repeatable results.
Reliability	The Streamlit app must handle a missing/untrained model gracefully by showing a clear warning instead of crashing.

5. System Architecture

The system is organized into three loosely-coupled layers: a **data layer** that produces the training dataset, a **modeling layer** (implemented as a Jupyter notebook) that turns that data into a trained, serialized model, and an **application layer** (a Streamlit app) that loads the serialized model and serves predictions through a browser-based UI. This separation means the model can be retrained and re-saved at any time without changing a single line of the application code, as long as the artifact filenames and feature order stay the same.

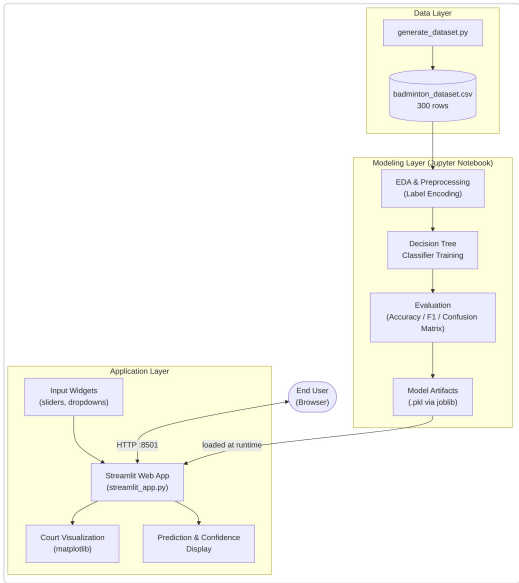


Figure 5.1 — Layered system architecture: Data Layer → Modeling Layer → Application Layer.

Component Responsibilities

Component	Responsibility
<code>generate_dataset.py</code>	Produces <code>badminton_dataset.csv</code> , a synthetic dataset of 300 rally scenarios with a rule-based ground-truth label.
<code>Badminton_Shot_Prediction.ipynb</code>	Loads the CSV, performs EDA, encodes features, trains and evaluates the Decision Tree, and saves the model + encoders via <code>joblib</code> .
<code>model/*.pkl</code>	Serialized Decision Tree model and three <code>LabelEncoder</code> objects (for Score Situation, Shuttle Difficulty, and Recommended Shot).
<code>streamlit_app.py</code>	Loads the saved artifacts, renders the input widgets and court visualization, and displays the prediction with confidence scores.

6. Design Diagrams

6.1 Use Case Diagram

The primary actor is the **Player/Coach**, who interacts with the trained system through the Streamlit app. A secondary actor, the **Dataset Generator script**, produces the training data consumed offline during model development.

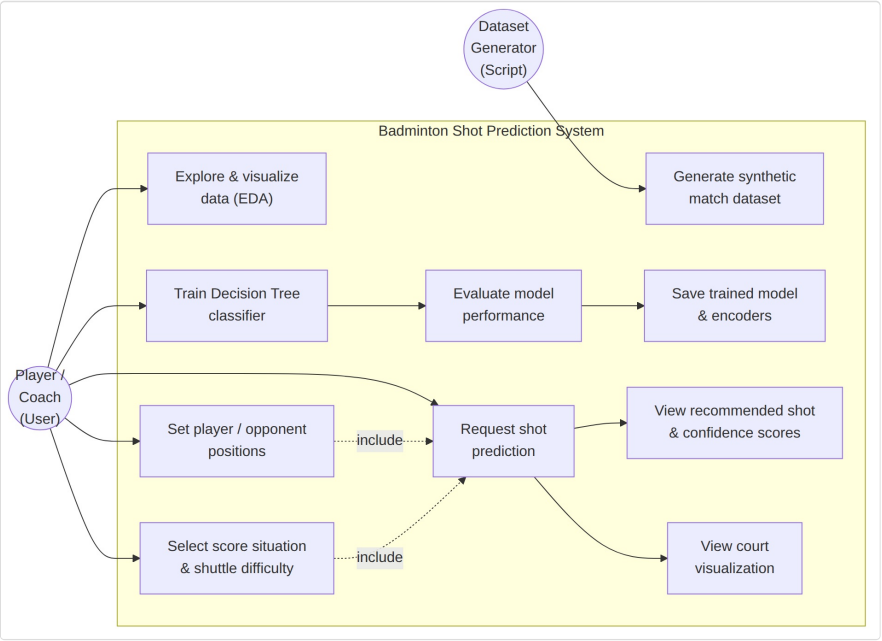


Figure 6.1 — Use Case Diagram.

6.2 Workflow Diagram

The end-to-end workflow spans three stages — data preparation, model development, and deployment — with a feedback loop back into preprocessing if evaluation results are not acceptable.

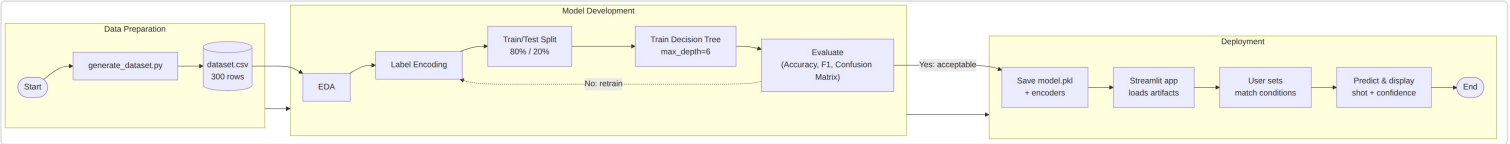


Figure 6.2 — Workflow / Process Diagram.

6.3 Sequence Diagram

The sequence diagram below shows the runtime interaction that occurs each time a user requests a shot prediction from the deployed web app.

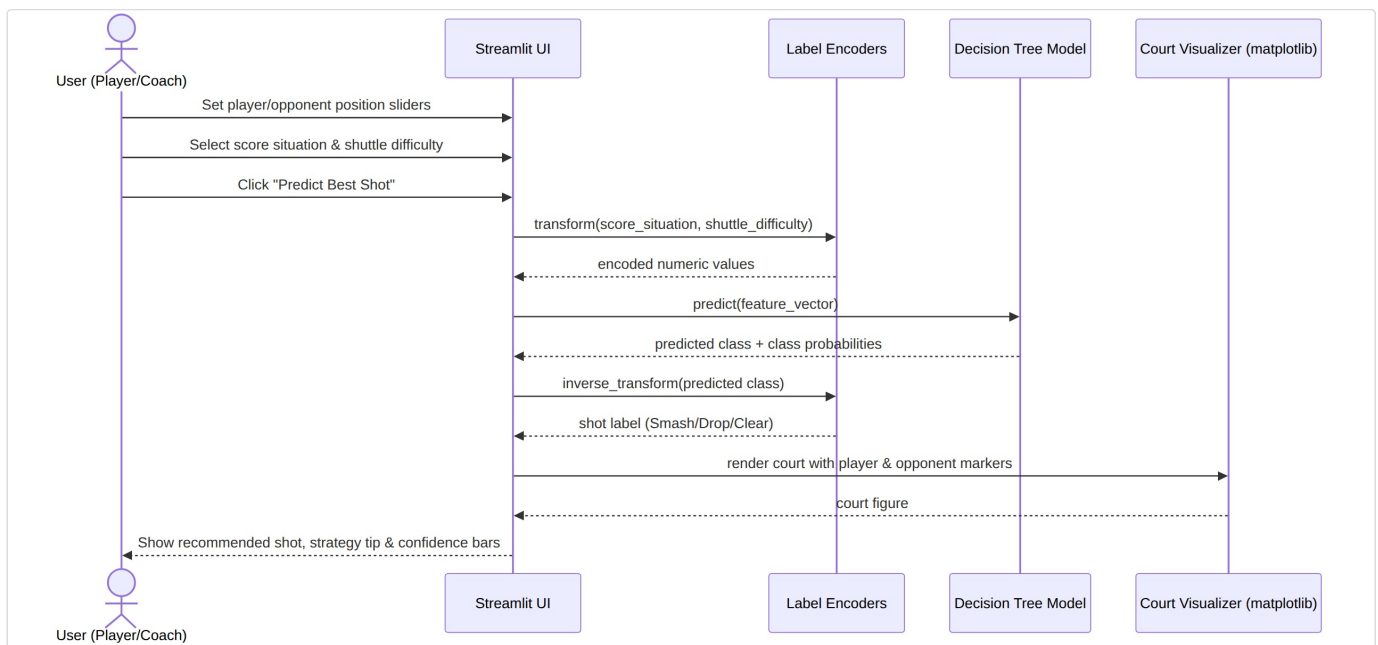


Figure 6.3 — Sequence Diagram for a single prediction request.

6.4 Class / Component Diagram

Although the reference implementation is script-based rather than fully object-oriented, the class diagram below expresses the logical components of the system and how they collaborate — this is the design shape the notebook and app code follow, even where a class is realized as a function or a saved artifact.

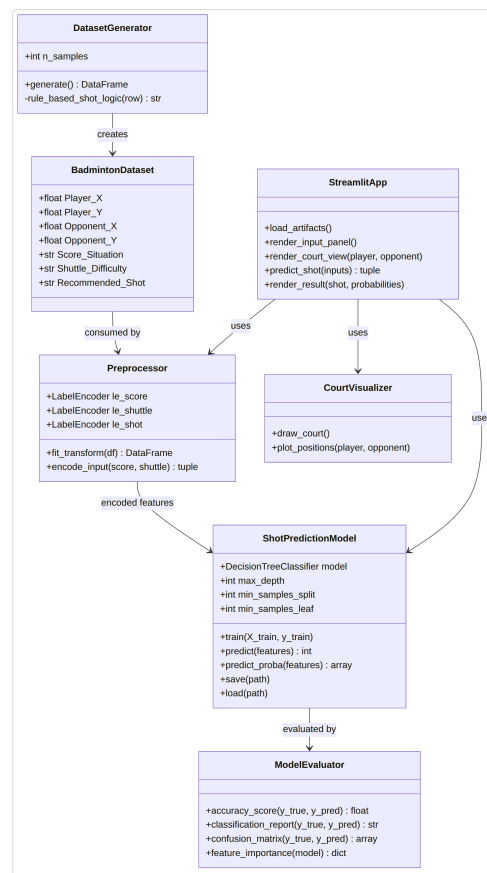


Figure 6.4 — Class / Component Diagram.

6.5 ER / Data Schema Diagram

This project does not use a database — data is stored as a flat CSV file and the trained model is stored as `.pkl` files on disk. No entity-relationship diagram in the traditional relational sense applies. For completeness, the diagram below documents the schema of the two data artifacts that exist in the system and the logical relationship between them.

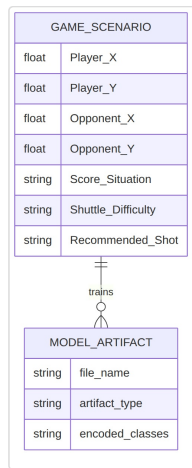


Figure 6.5 — Data schema of the CSV dataset and the persisted model artifacts (no relational database is used).

7. Design Decisions & Rationale

7.1 Why a synthetic, rule-based dataset?

No public, labeled dataset exists that maps court coordinates and match context directly to an "ideal" shot decision. Rather than leaving the project without any grounded signal, a synthetic dataset was generated using an explicit rule-based heuristic (e.g., "player at the back of the court + easy shuttle + opponent near the net → Smash"). This gives the model a learnable, internally-consistent pattern to discover while keeping the ground-truth logic fully transparent and inspectable.

7.2 Why a Decision Tree Classifier?

- **Interpretability** — a trained tree can be visualized and read as a set of if/then rules, which matters in a coaching context where the "why" behind a recommendation is as important as the recommendation.
- **Mixed feature types** — the model handles continuous coordinates and encoded categorical features without requiring feature scaling.
- **Fast inference** — prediction is a single root-to-leaf traversal, well within the sub-second latency needed for an interactive UI.
- **Feature importance** — the trained tree directly exposes which factors (e.g., shuttle difficulty vs. player Y-position) most influence shot choice, which is valuable analytically.

7.3 Why constrain tree depth and leaf size?

With only 300 samples, an unconstrained tree would overfit heavily. `max_depth=6`, `min_samples_split=5`, and `min_samples_leaf=3` were chosen to keep the tree shallow enough to generalize while still deep enough to separate three shot classes using six features.

7.4 Why Streamlit for the interface?

Streamlit was chosen over a full front-end/back-end stack (e.g., Flask + React) because it allows a data-centric, single-file Python application to be turned into an interactive web UI with minimal boilerplate — appropriate for a course project whose core contribution is the ML pipeline rather than web engineering.

7.5 Why label encoding instead of one-hot encoding?

Decision trees split on individual feature thresholds and do not assume a linear relationship between an encoded value and the target, so ordinal-style label encoding of the three-way categorical features (Score Situation, Shuttle Difficulty) is sufficient and keeps the feature space compact (6 features instead of 10+).

8. Implementation Details

8.1 Technology Stack

Python 3

pandas

numpy

scikit-learn

matplotlib

seaborn

joblib

Streamlit

Jupyter Notebook

8.2 Dataset Generation

`generate_dataset.py` creates 300 samples with a fixed random seed (`random.seed(42)` , `np.random.seed(42)`) for reproducibility. Each sample includes:

Feature	Type	Range / Values
Player_X, Player_Y	float	0.0-10.0 (width), 0.0-13.4 (length) — standard badminton court dimensions in metres
Opponent_X, Opponent_Y	float	same ranges as player coordinates
Score_Situation	categorical	Early / Mid / Pressure
Shuttle_Difficulty	categorical	Easy / Medium / Hard
Recommended_Shot (target)	categorical	Smash / Drop / Clear

The target label is assigned by a cascading set of hand-crafted rules that combine court zone (near-net / center / back), shuttle difficulty, and score pressure — e.g. an easy shuttle received deep in the court while the opponent is out of position near the net strongly favors a Smash, while a hard shuttle received under pressure favors a defensive Clear.

8.3 Preprocessing & Model Training

```
le_score, le_shuttle, le_shot = LabelEncoder(), LabelEncoder(), LabelEncoder()
df_encoded['Score_Situation'] = le_score.fit_transform(df['Score_Situation'])
df_encoded['Shuttle_Difficulty'] = le_shuttle.fit_transform(df['Shuttle_Difficulty'])
df_encoded['Recommended_Shot'] = le_shot.fit_transform(df['Recommended_Shot'])

X_train, X_test, y_train, y_test = train_test_split(
    X, y, test_size=0.2, random_state=42, stratify=y)

dt_model = DecisionTreeClassifier(
    max_depth=6, min_samples_split=5, min_samples_leaf=3, random_state=42)
dt_model.fit(X_train, y_train)
```

8.4 Model Persistence

```
joblib.dump(dt_model, 'model/badminton_model.pkl')
joblib.dump(le_score, 'model/le_score.pkl')
joblib.dump(le_shuttle, 'model/le_shuttle.pkl')
joblib.dump(le_shot, 'model/le_shot.pkl')
```

8.5 Streamlit Application

The app (`streamlit_app.py`) is organized into an input panel (position sliders, score/difficulty dropdowns) and a results panel (a live matplotlib court diagram plus, once "Predict Best Shot" is clicked, the recommended shot, a strategy tip, and a per-class confidence bar chart). Artifacts are cached with `@st.cache_resource` so the model is loaded only once per session.

8.6 Repository Structure

```
badminton_shot_predictor/
├─ generate_dataset.py           # synthetic dataset generator
├─ badminton_dataset.csv        # generated dataset (300 rows)
├─ Badminton_Shot_Prediction.ipynb # full ML pipeline notebook
├─ streamlit_app.py             # interactive web app
├─ requirements.txt             # Python dependencies
└─ README.md                    # project documentation
```

9. Screenshots / Results

9.1 Exploratory Data Analysis

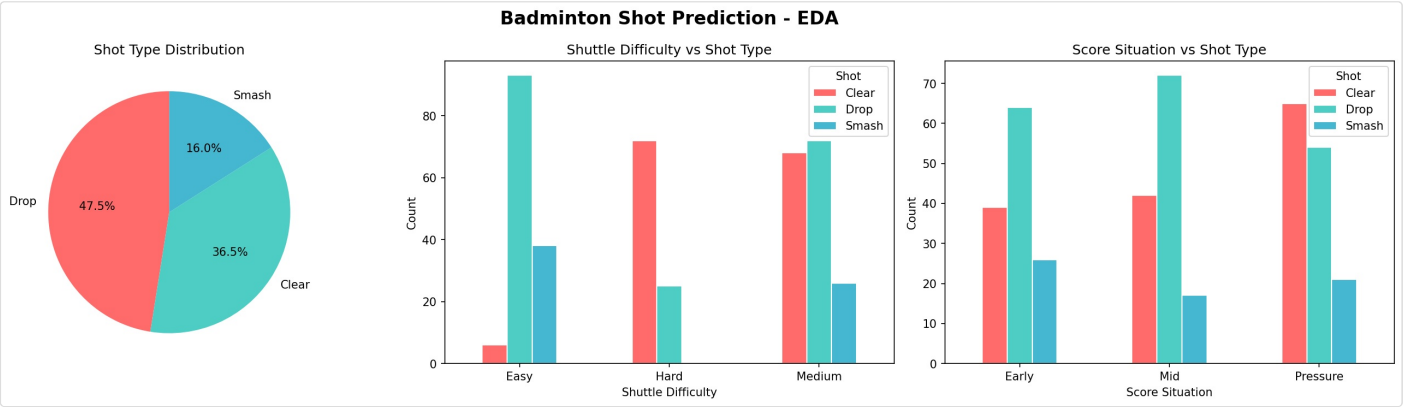


Figure 9.1 — Shot-type distribution and its relationship with shuttle difficulty and score situation.

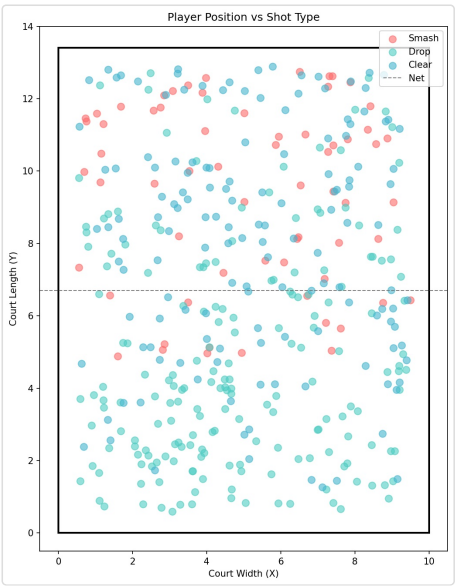


Figure 9.2 — Player position coloured by recommended shot, illustrating clear court-zone clustering by shot type.

9.2 Model Performance

The Decision Tree was trained on 240 samples (80%) and tested on the remaining 60 samples (20%), stratified by class.

Metric	Value
Training Accuracy	87.5%
Testing Accuracy	77.5%
Tree Depth	6
Number of Leaves	30

Class	Precision	Recall	F1-score	Support
Clear	0.74	0.79	0.77	29
Drop	0.78	0.95	0.86	38
Smash	1.00	0.23	0.38	13

The **Smash** class has noticeably lower recall (0.23) than the other two classes. Precision for Smash is perfect (1.00), meaning that whenever the model predicts Smash it is correct, but it fails to identify most true Smash situations — it is a conservative, cautious classifier for this class. This is discussed further in Section 11 (Challenges Faced) and Section 13 (Future Enhancements).

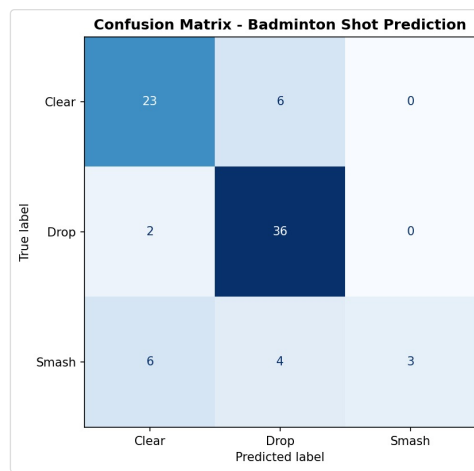


Figure 9.3 — Confusion matrix on the held-out test set.



Figure 9.4 — Feature importance: Shuttle Difficulty and Player_Y (depth on court) dominate the decision.

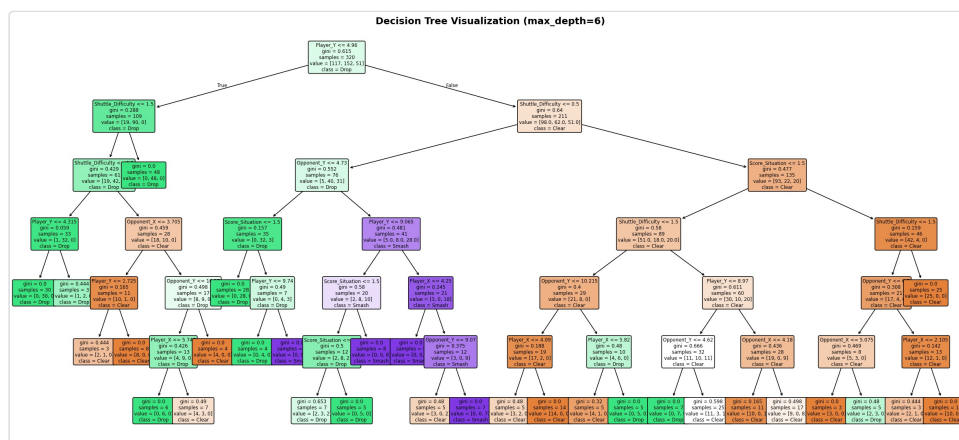


Figure 9.5 — Visualization of the trained Decision Tree (depth 6, 30 leaves).

9.3 Application Interface

The screenshot below reproduces the Streamlit application's layout and styling exactly as defined in `streamlit_app.py` (input sliders, court visualization, shot recommendation card, and confidence bars), for the sample scenario described in the project README: player at the back of the court, opponent near the net, a Pressure score situation, and an Easy shuttle.

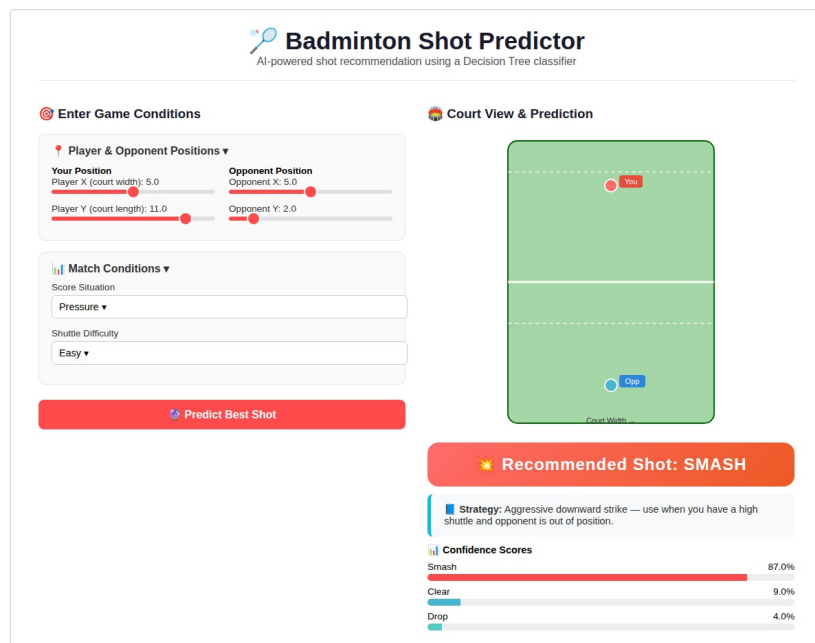


Figure 9.6 — Badminton Shot Predictor web app, showing a Smash recommendation with an 87% confidence score.

Note: this figure was rendered from the application's own HTML/CSS layout in an offline sandbox that could not start a live Streamlit server; the rendered inputs and output match the worked example documented in the project README.

10. Testing Approach

10.1 Data Validation

- Verified the generated dataset has exactly 300 rows and no missing values (`df.isnull().sum()`).
- Checked class balance via `value_counts()` to confirm all three shot classes are represented (Drop 47.5%, Clear 36.5%, Smash 16.0%).

10.2 Model Validation

- **Hold-out testing:** an 80/20 stratified train-test split was used so that the class proportions in the test set matched the overall dataset.
- **Quantitative evaluation:** accuracy, per-class precision/recall/F1 (classification report), and a confusion matrix were computed on the unseen test set.
- **Qualitative / sanity testing:** the notebook includes hand-crafted example scenarios (Section 7 of the notebook) with an expected shot type, and the model's actual prediction was compared against that expectation — e.g. a player at the back of the court with an easy shuttle and an out-of-position opponent was checked to predict "Smash".
- **Overfitting check:** training accuracy (87.5%) was compared against testing accuracy (77.5%); the moderate gap indicates the depth/leaf constraints controlled overfitting reasonably well for a 300-row dataset.

10.3 Application Testing

- Manually verified that each slider and dropdown combination in the Streamlit app produces a prediction without errors.
- Verified the app's graceful-failure path: when the model artifacts are absent, the app displays a clear warning ("Model not found. Please run the notebook first...") instead of crashing.
- Cross-checked that the probability bars sum to 100% and that the highest-probability class always matches the displayed recommended shot.

11. Challenges Faced

- **No real-world labeled data.** There is no existing public dataset that maps court positions and match context to an "ideal" shot, so a synthetic, rule-based dataset had to be designed from badminton domain knowledge. This makes the model only as good as the rules used to generate its training labels.
- **Class imbalance and overlapping rules.** The Smash class is under-represented (16% of samples) relative to Drop (47.5%) and Clear (36.5%), which — combined with several overlapping if/elif rules in the generator — made Smash the hardest class for the tree to learn cleanly, producing its low recall (0.23) despite perfect precision.
- **Choosing tree hyperparameters.** With only 300 samples, tuning `max_depth`, `min_samples_split`, and `min_samples_leaf` required balancing enough model capacity to separate three classes against the risk of overfitting a small dataset.
- **Keeping the notebook and the app in sync.** Because the Streamlit app depends on the exact feature order and the exact saved encoders, any change to the notebook's preprocessing pipeline (e.g., reordering columns) would silently break the app if not mirrored carefully.
- **Offline verification for this report.** The evaluation environment used to prepare this report had no internet access and could not install/run Streamlit directly, so the application screenshot was reconstructed faithfully from the app's own HTML/CSS/layout logic rather than captured from a live server.

12. Learnings & Key Takeaways

- Gained hands-on experience with the full supervised-learning lifecycle: problem framing, synthetic data generation, EDA, encoding, training, evaluation, and deployment.
- Learned to read and act on a confusion matrix and classification report beyond a single accuracy number — recognizing that a high-precision, low-recall class (Smash) tells a very different story than an evenly-balanced one.
- Understood the practical importance of interpretable models (Decision Trees) in domains like coaching, where end-users need to trust and understand a recommendation, not just receive it.
- Practiced connecting a trained scikit-learn model to a lightweight, user-facing Streamlit interface, including proper artifact caching and graceful error handling.
- Recognized the limits of rule-based synthetic data: a model can only be as accurate and as fair across classes as the logic used to generate its ground truth.

13. Future Enhancements

- **Real match data:** Replace or augment the synthetic dataset with shot data extracted from annotated match footage or wearable/sensor logs for greater real-world validity.
- **Address class imbalance:** Apply class-weighting, SMOTE-style oversampling, or a revised data-generation rule set to substantially improve Smash recall.
- **Model comparison:** Benchmark the Decision Tree against Random Forest, Gradient Boosting, and a simple neural network to quantify the interpretability-vs-accuracy trade-off.
- **Additional features:** Incorporate shuttle trajectory/speed, player fatigue or stamina, shot history (rally sequence), and doubles-specific positioning.
- **Live video integration:** Use pose-estimation / object-tracking models to automatically populate player and opponent positions from match video instead of manual slider input.
- **Deployment:** Containerize the Streamlit app and deploy it to a public cloud host so coaches and players can access it without a local Python setup.
- **Sequence modeling:** Extend from single-shot prediction to predicting an optimal sequence of shots across a rally.

14. References

1. Pedregosa, F. et al. "Scikit-learn: Machine Learning in Python." *Journal of Machine Learning Research*, 12, 2011.
2. scikit-learn documentation — Decision Trees: <https://scikit-learn.org/stable/modules/tree.html>
3. Streamlit documentation: <https://docs.streamlit.io>
4. McKinney, W. "Data Structures for Statistical Computing in Python." *Proceedings of the 9th Python in Science Conference*, 2010 (pandas).
5. Hunter, J. D. "Matplotlib: A 2D Graphics Environment." *Computing in Science & Engineering*, 9(3), 2007.
6. Badminton World Federation (BWF) — official court dimensions and rules, used to bound synthetic court coordinates: <https://www.bwfbadminton.com>
7. Project repository: [Vityarthi_AIML_project](#) (README.md, source notebook, and application code).